

C++

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to C++

- 2. Variables and Data Types

★ 3. Input and Output (cin, cout)

- 4. Conditional Statements (if, else if, else)

★ 5. Loops - for, while, do-while

- 6. Arrays

★ 7. Strings (std::string)

★ 8. Functions

- 9. Function Overloading

★ 10. Pointers and References

★ 11. Classes and Objects

★ 12. Constructors and Destructors

★ 13. Inheritance

★ 14. Polymorphism (Virtual Functions)

- 15. Encapsulation
- 16. Abstraction (Abstract Class)
- 17. Operator Overloading
- 18. Templates

★ 19. Exception Handling (try, catch)

★ 20. STL - Vector

- 21. STL - Map

★ 1. Introduction to C++ [IMPORTANT TOPIC]

Explanation:

C++ is an upgraded, more powerful version of the C language. It adds Object-Oriented Programming (OOP) features to C, like classes and objects, while still keeping C's speed and power.

C++ is widely used to build games, operating systems, and high-performance software.

Key Points:

- Supports both procedural (step-by-step) and object-oriented programming.
- Faster than many other languages because it runs very close to the hardware.
- Every C++ program must have a main() function.
- Uses cin and cout for input and output.

Example Code:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

Output:

Hello, World!

□ *Note: using namespace std; lets us write cout instead of std::cout every time.*

2. Variables and Data Types

Explanation:

A variable stores a value in memory. Like C, C++ requires us to declare the data type of a variable before we use it.

Key Points:

- int -> whole numbers
- float / double -> decimal numbers
- char -> a single character
- bool -> true or false
- string -> text (needs #include <string>)

Example Code:

```
#include <iostream>
using namespace std;

int main() {
    string name = "Pravendra";
    int age = 21;
    cout << name << " is " << age << " years old";
    return 0;
}
```

Output:

Pravendra is 21 years old

★ 3. Input and Output (cin, cout) *[IMPORTANT TOPIC]*

Explanation:

cout (pronounced 'see-out') is used to print output on the screen, using the << symbol. cin (pronounced 'see-in') is used to take input from the user, using the >> symbol.

Example Code:

```
int age;  
cout << "Enter your age: ";  
cin >> age;  
cout << "You are " << age << " years old";
```

Output:

```
Enter your age: 21  
You are 21 years old
```

4. Conditional Statements (if, else if, else)

Explanation:

Conditional statements let the program make decisions based on whether a condition is true or false.

Example Code:

```
int marks = 75;

if (marks >= 90) {
    cout << "Grade A";
} else if (marks >= 60) {
    cout << "Grade B";
} else {
    cout << "Grade C";
}
```

Output:

Grade B

★ 5. Loops - for, while, do-while [IMPORTANT TOPIC]

Explanation:

Loops repeat a block of code multiple times. 'for' is used when we know how many times to repeat. 'while' repeats as long as a condition is true. 'do-while' runs at least once, even if the condition is false.

Example Code:

```
for (int i = 1; i <= 3; i++) {  
    cout << "For loop: " << i << endl;  
}
```

Output:

```
For loop: 1  
For loop: 2  
For loop: 3
```

□ Note: *endl moves the output to a new line, similar to \n.*

6. Arrays

Explanation:

An array stores multiple values of the same data type together in one variable, with a fixed size, accessed using an index starting from 0.

Example Code:

```
int numbers[4] = {10, 20, 30, 40};  
cout << numbers[0] << endl;  
for (int i = 0; i < 4; i++) {  
    cout << numbers[i] << " ";  
}
```

Output:

```
10  
10 20 30 40
```

★ 7. Strings (std::string) [IMPORTANT TOPIC]

Explanation:

Unlike C, C++ has a proper built-in String type, called 'string', which is much easier to use than character arrays. It comes with many ready-made functions.

Example Code:

```
#include <string>

string name = " Pravendra ";
cout << name.length() << endl;
cout << name + " Sharma";
```

Output:

```
13
 Pravendra  Sharma
```

□ *Note: In C++, you can join (concatenate) two strings directly using the + symbol.*

★ 8. Functions [IMPORTANT TOPIC]

Explanation:

A function is a reusable block of code that performs a specific task. We write it once and call it whenever needed, instead of repeating the same code.

Example Code:

```
int addNumbers(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int total = addNumbers(5, 7);  
    cout << "Total: " << total;  
    return 0;  
}
```

Output:

Total: 12

9. Function Overloading

Explanation:

Function overloading means writing multiple functions with the SAME name but different parameters. C++ automatically picks the correct one based on what you pass to it.

Example Code:

```
int add(int a, int b) { return a + b; }  
double add(double a, double b) { return a + b; }  
  
cout << add(2, 3) << endl;           // uses int version  
cout << add(2.5, 3.5);               // uses double version
```

Output:

```
5  
6
```

★ 10. Pointers and References *[IMPORTANT TOPIC]*

Explanation:

A pointer stores the memory address of another variable. A reference is like a nickname for an existing variable — both point to the same memory, but a reference is often easier and safer to use.

Example Code:

```
int age = 21;
int *ptr = &age;    // pointer
int &ref = age;      // reference

cout << *ptr << endl; // 21
ref = 25;
cout << age;          // 25 (changed through the reference)
```

Output:

```
21
25
```

★ 11. Classes and Objects *[IMPORTANT TOPIC]*

Explanation:

A class is a blueprint used to create objects, which group data (called member variables) and behavior (called member functions) together. An object is a real item made from that class blueprint.

Example Code:

```
class Car {  
    public:  
        string brand;  
        void showBrand() {  
            cout << "Brand: " << brand;  
        }  
};  
  
Car c1;  
c1.brand = "Toyota";  
c1.showBrand();
```

Output:

Brand: Toyota

★ 12. Constructors and Destructors [IMPORTANT TOPIC]

Explanation:

A constructor is a special function that runs automatically when an object is created, usually to set starting values. A destructor runs automatically when an object is destroyed, usually to clean up resources. A destructor's name starts with a ~ symbol.

Example Code:

```
class Student {  
    public:  
        string name;  
        Student(string n) {    // constructor  
            name = n;  
            cout << "Object created for " << name << endl;  
        }  
        ~Student() {           // destructor  
            cout << "Object destroyed" << endl;  
        }  
};  
  
Student s1("Pravendra");
```

Output:

```
Object created for Pravendra  
Object destroyed (runs automatically at the end)
```

★ 13. Inheritance *[IMPORTANT TOPIC]*

Explanation:

Inheritance lets one class (derived/child class) reuse the properties and functions of another class (base/parent class), using a colon ':' symbol. This avoids rewriting the same code again.

Example Code:

```
class Animal {  
    public:  
        void eat() { cout << "This animal eats food" << endl; }  
};  
  
class Dog : public Animal {  
    public:  
        void bark() { cout << "The dog barks"; }  
};  
  
Dog d;  
d.eat();  
d.bark();
```

Output:

```
This animal eats food  
The dog barks
```

★ 14. Polymorphism (Virtual Functions) [IMPORTANT TOPIC]

Explanation:

Polymorphism means 'many forms' — the same function name behaves differently for different objects. In C++, this is achieved using 'virtual' functions, which allow a child class to properly override (replace) a parent class's function.

Example Code:

```
class Animal {
public:
    virtual void sound() { cout << "Animal makes a sound"; }
};

class Cat : public Animal {
public:
    void sound() override { cout << "Cat says Meow"; }
};

Animal *a = new Cat();
a->sound();
```

Output:

Cat says Meow

□ *Note: Marking the parent function 'virtual' is what allows real polymorphism to work correctly in C++.*

15. Encapsulation

Explanation:

Encapsulation means hiding a class's internal data using 'private', and only allowing access to it through public functions (getters and setters). This protects the data from being changed incorrectly from outside the class.

Example Code:

```
class BankAccount {  
    private:  
        double balance;  
    public:  
        void setBalance(double amount) { balance = amount; }  
        double getBalance() { return balance; }  
};  
  
BankAccount acc;  
acc.setBalance(5000);  
cout << acc.getBalance();
```

Output:

5000

16. Abstraction (Abstract Class)

Explanation:

Abstraction means showing only the essential details and hiding the complex working. In C++, this is done using an abstract class — a class with at least one 'pure virtual function' (a function with no body, marked with = 0).

Example Code:

```
class Shape {  
    public:  
        virtual void draw() = 0;    // pure virtual function  
};  
  
class Circle : public Shape {  
    public:  
        void draw() override { cout << "Drawing a circle"; }  
};  
  
Shape *s = new Circle();  
s->draw();
```

Output:

Drawing a circle

17. Operator Overloading

Explanation:

Operator overloading lets us redefine how a normal operator (like +) works for objects of our own class. This makes objects easier to use, almost like normal numbers.

Example Code:

```
class Point {  
    public:  
        int x;  
        Point(int a) { x = a; }  
        Point operator+(Point p) {  
            return Point(x + p.x);  
        }  
};  
  
Point p1(5), p2(10);  
Point p3 = p1 + p2;    // uses our custom + operator  
cout << p3.x;
```

Output:

15

18. Templates

Explanation:

A template lets us write ONE function or class that works with ANY data type (int, double, string, etc.), instead of writing separate versions for each type.

Example Code:

```
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

cout << findMax(5, 10) << endl;           // works with int
cout << findMax(2.5, 1.5);                 // works with double
```

Output:

```
10
2.5
```

★ 19. Exception Handling (try, catch) [IMPORTANT TOPIC]

Explanation:

Exception handling lets us safely 'catch' an unexpected error while the program is running, instead of the whole program crashing suddenly.

Example Code:

```
try {  
    int a = 10, b = 0;  
    if (b == 0) throw "Cannot divide by zero!";  
    cout << a / b;  
} catch (const char* msg) {  
    cout << "Error: " << msg;  
}
```

Output:

Error: Cannot divide by zero!

□ Note: 'throw' is used to raise (create) an error on purpose, which is then caught by 'catch'.

★ 20. STL - Vector *[IMPORTANT TOPIC]*

Explanation:

A vector is part of the Standard Template Library (STL). It works like an array, but it can automatically grow or shrink in size. It is one of the most used data structures in real C++ programs.

Example Code:

```
#include <vector>

vector<int> numbers = {10, 20, 30};
numbers.push_back(40);

for (int n : numbers) {
    cout << n << " ";
}
cout << endl << numbers.size();
```

Output:

```
10 20 30 40
4
```

21. STL - Map

Explanation:

A map stores data as 'key : value' pairs, similar to a dictionary. It automatically keeps the keys sorted and makes it easy to find a value using its key.

Example Code:

```
#include <map>

map<string, int> marks;
marks["Pravendra"] = 90;
marks["Aman"] = 85;

cout << marks["Pravendra"];
```

Output:

90